EE538 Final Project Proposal

Paged KV Cache Manager with Radix Prefix Sharing

Track B: LLM/MLSys module | November 5, 2026

1 People and repository

Member	Contact	Responsibility
Bohan Cui USC ID 1639422176	bohancui@usc.edu	Block pool and tables, reference counts, copy-on-write, template heap
Bowen Deng USC ID：2007371783	bowenden@usc.edu	Radix prefix index, LRU and FIFO eviction
Yingtao Yang USC ID:5313792026	yingtaoy@usc.edu	Manager integration, CLI, reference model and experiments; integration lead

Team GitHub repository: https://github.com/ryan63-ryan/Final_Project/blob/main

2 Problem and interface

We will build a C++ manager that stores request payloads in a bounded pool of fixed-size blocks. Requests can reuse full prefix blocks through a radix index; forks can also share partial blocks until a write requires a private copy. Deterministic bytes derived from each token prefix stand in for KV tensors, allowing storage and isolation checks without a model [1, 2].

The CLI reads a configuration and a trace file. Commands are ADMIT req tokens..., APPEND req token, FORK new src, RELEASE req and STATS. Output contains operation status, block IDs, cache hits and summary metrics. Missing files, malformed lines and invalid request IDs produce defined errors.

Example with two tokens per block and capacity four blocks:

```
ADMIT r1 10 11 12  -> OK blocks=[0,1] cached=0
FORK r2 r1        -> OK blocks=[0,1]
APPEND r2 13      -> OK blocks=[0,2] cow=1
RELEASE r1        -> OK freed=[1]
```

3 Scope and behavior

The first prototype will support independent allocation and admit/append/release. Phase II adds whole-block prefix matching, radix edge splits, fork, copy-on-write, LRU/FIFO eviction and tested request lifecycles. Only full blocks enter the prefix cache. APPEND allocates a block when the tail is full, or copies a shared partial tail before writing.

RELEASE decrements reference counts and frees a partial block only at zero references. Indexed full blocks remain cached; only unpinned leaf blocks may be evicted. Admission checks free plus reclaimable capacity first, preserving cache and request state on rejection. Tests cover shared releases, divergence, cascading eviction and exhausted capacity. GPU kernels, models, threading and distributed serving are outside scope.

4 Course mapping and design

Our custom structures are a radix tree, an indexed MinHeap<T, Compare>, and a block pool with a free list. Algorithms are longest-prefix matching with edge splits, copy-on-write with reference counting, and greedy leaf eviction. Compressed radix edges hold whole token chunks and their BlockIds; matching and splitting use block boundaries.

BlockPool owns bytes through RAII and disables copying; BlockTable maps each request to blocks. RadixTree owns nodes with unique_ptr. KVCacheManager coordinates them. LRUPolicy and FIFOPolicy implement an abstract EvictionPolicy through virtual calls. STL containers provide supporting storage. Core logic is our own; papers supply design ideas, and Python supports data generation and plots.

5 Correctness

Tests check heap order, pool accounting, valid prefix links, request reference counts, and exclusion of pinned blocks from eviction. Normal and edge cases include empty prompts, exact block boundaries, fork divergence, parent-first release, invalid commands and failed allocation. A small golden trace has complete input and expected output.

An independent vector model checks accepted requests' bytes; brute-force scans of small expected cache histories check whole-block prefix hits. Hand-calculated traces check capacity, rejection and eviction decisions. The no-sharing baseline may make different admission decisions. Seeded traces and AddressSanitizer/UndefinedBehaviorSanitizer check lifetime errors. If unavailable, we document this and use allocation counters and repeated lifecycle tests.

6 Evaluation

We ask when sharing benefits outweigh its overhead and how LRU/FIFO handles pressure. Sharing off/on and LRU/FIFO replay identical traces at the same physical capacity. Later events for rejected requests are skipped consistently; rejection rates and completed operations accompany latency results.

We test 1,000, 10,000 and 100,000 requests. Two factors vary: prefix sharing (unique prompts, shared system prefixes, multi-turn prefixes) and capacity (256, 1,024, 4,096 blocks). Blocks hold 16 tokens of 64 payload bytes each. Seeded traces bound prompt lengths and active requests, covering low pressure and saturation. We use selected sweeps, not all parameter combinations.

Metrics cover hit rate, uncached prompt tokens as a recomputation proxy, blocks used, fragmentation, copy counts, latency and memory. We separate operation timing from parsing, checks and driver overhead. Each setting runs five times; we report the median, interquartile range, hardware, compiler flags and seed. Memory includes peak RSS where available and explicit storage estimates. Raw CSVs and scripts reproduce plots. Main-operation time/space analysis accompanies results; token counts do not establish model-serving speedups.

7 Resources

Resources are laptops with 8 GB RAM, C++17 with compiler warnings, GCC/Clang, CMake and Python/NumPy/Matplotlib. Default pools use at most 4 MiB; traces and results have a 1 GB disk budget. Synthetic data needs no external access or payment. Smaller generated traces are the fallback for resource limits; GPU hardware is unnecessary.

8 Plan and risk

Dates	Milestone
Nov 9-15	After scope approval: freeze interfaces; build the reference, driver and independent-allocation prototype.
Nov 16-22	Integrate prefix sharing, fork, copy-on-write and eviction; complete core tests and functionality.
Nov 23-29	Run controlled experiments, fix defects and explain results.
Nov 30-Dec 2	Reproduce from a clean checkout; finish README, report and demo; tag the submitted version.

All members test their modules and contribute to the report and demo; Yingtao leads integration. Reference-count errors across split, fork and release are the main risk, addressed by early small tests and debug invariants. Extra parameter sweeps are stretch goals; Phase III is not planned.

References

[1] Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023, Sections 4.2-4.4. doi:10.1145/3600006.3613165.
[2] Zheng et al. SGLang: Efficient Execution of Structured Language Model Programs. NeurIPS 2024, Section 3.
[3] EE538 Fall 2026 Final Project Assignment.